

MMFileTransfer

Installation Without Docker (Linux Server)

This guide installs MMFileTransfer directly on a Linux server **without Docker**.

Internet: the server needs internet **only during installation** (to download the software). Once installed, the app **runs fully offline** — it never needs internet again. The only thing that needs the internet is the install step below.

⚡ Fast path — one command

If you just want it installed, copy the repo to the server and run:

```
sudo bash deploy/install_no_docker.sh
```

It auto-detects the server IP, generates strong passwords/secrets, adds swap, and starts everything — printing the admin login at the end. (Override the IP with `sudo SERVER_IP=10.20.30.40 bash deploy/install_no_docker.sh`.) Targets Ubuntu/Debian. The manual steps below explain what that script does, for understanding, customization, or RHEL.

The app is not a single program — it is **6 services** that all run at once and talk to each other over `localhost`:

Service	Role	Runs as
PostgreSQL	database (users, file metadata, audit)	OS service
Redis	Celery broker + Django cache	OS service
MinIO	file storage (files live in a folder on this disk)	systemd service
Gunicorn	the Django web app	<code>mmftp-web.service</code>
Celery worker	background jobs (email, file activation, cleanup)	<code>mmftp-worker.service</code>
Celery beat	scheduler (daily expiry/retention)	<code>mmftp-beat.service</code>
nginx	reverse proxy + serves downloads	OS service

MinIO stores files in a plain local folder on this server. It is not a cloud service. The files never leave the box.

Folder layout

You create one main folder, `/opt/mmftp`, with these inside:

```
/opt/mmftp/
├─ app/           ← the application CODE
├─ venv/          ← Python + all the packages
├─ minio-data/    ← empty at first; MinIO stores uploaded files here
└─ .env           ← configuration (holds secrets – chmod 600)
```

A few pieces go to standard system locations:

```
/usr/local/bin/minio , /usr/local/bin/mc  ← the MinIO program + admin tool
/etc/systemd/system/*.service             ← the service files (auto-start)
/etc/nginx/sites-available/mmftp           ← the nginx config
```

STEP 1 — Install the system software (needs internet)

Ubuntu / Debian:

```
sudo apt update
sudo apt install -y python3.12 python3.12-venv postgresql redis-server nginx libmagic1
sudo systemctl enable --now postgresql redis-server nginx
```

RHEL / Rocky / Alma:

```
sudo dnf install -y python3.12 postgresql-server redis nginx file-utils
sudo postgresql-setup --initdb
sudo systemctl enable --now postgresql redis nginx
```

STEP 2 — Put the code on the server

```
sudo mkdir -p /opt/mmftp/{app,minio-data}
sudo chown -R $USER:$USER /opt/mmftp

# Copy the code in however you like – clone it, or copy the folder across:
git clone <your-repo-url> /opt/mmftp/app
```

Make sure the migration files are committed in the code — the app applies committed migrations; it does not generate them here.

STEP 3 — Python environment + packages (needs internet)

```
python3.12 -m venv /opt/mmftp/venv
/opt/mmftp/venv/bin/pip install -r /opt/mmftp/app/requirements.txt
```

STEP 4 — PostgreSQL: create the database and user

```
sudo -u postgres psql <<'SQL'
CREATE USER mmftp WITH PASSWORD 'CHOOSE_A_DB_PASSWORD';
CREATE DATABASE mmftp OWNER mmftp;
SQL
```

Use the **same** password in the `.env` (Step 6).

STEP 5 — MinIO: install the binary and run it as a service (needs internet)

```
sudo curl -L https://dl.min.io/server/minio/release/linux-amd64/minio -o /usr/local/bin/minio
sudo curl -L https://dl.min.io/client/mc/release/linux-amd64/mc -o /usr/local/bin/mc
sudo chmod +x /usr/local/bin/minio /usr/local/bin/mc
```

Create `/etc/systemd/system/minio.service` (pick your own access/secret keys — the secret must be at least 8 characters):

```

[Unit]
Description=MinIO object storage
After=network.target

[Service]
User=root
Environment=MINIO_ROOT_USER=CHOOSE_A_MINIO_KEY
Environment=MINIO_ROOT_PASSWORD=CHOOSE_A_MINIO_SECRET
ExecStart=/usr/local/bin/minio server /opt/mmftp/minio-data --address 127.0.0.1:9000 --console-address 127.0.0.1:9001
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

Start it and create the bucket (use the same keys you just set):

```

sudo systemctl daemon-reload
sudo systemctl enable --now minio

mc alias set local http://127.0.0.1:9000 CHOOSE_A_MINIO_KEY CHOOSE_A_MINIO_SECRET
mc mb --ignore-existing local/mmftp-files

```

STEP 6 — Configuration: `/opt/mmftp/.env`

First generate the two secrets:

```

/opt/mmftp/venv/bin/python -c "import secrets;print(secrets.token_urlsafe(64))" # DJANGO_SECRET_KEY
/opt/mmftp/venv/bin/python -c "from cryptography.fernet import Fernet;print(Fernet.generate_key().decode())" #
SECRETS_ENCRYPTION_KEY

```

Create `/opt/mmftp/.env`. Below is a **filled-in example** assuming the server IP is `192.168.1.50` — replace it with your real IP/hostname:

```
# --- Django ---
DJANGO_SECRET_KEY=PASTE_THE_64_CHAR_SECRET_HERE
DJANGO_DEBUG=False
DJANGO_TIME_ZONE=Asia/Kolkata
DJANGO_LOG_LEVEL=INFO
DJANGO_SKIP_MAKEMIGRATIONS=1

DJANGO_ALLOWED_HOSTS=192.168.1.50,localhost,127.0.0.1,[:,1]
CSRF_TRUSTED_ORIGINS=http://192.168.1.50

SECRETS_ENCRYPTION_KEY=PASTE_THE_FERNET_KEY_HERE

TRUSTED_PROXY_IPS=127.0.0.1,::1
DJANGO_SECURE_SSL=False

# --- PostgreSQL (same machine → 127.0.0.1) ---
POSTGRES_DB=mmftp
POSTGRES_USER=mmftp
POSTGRES_PASSWORD=CHOOSE_A_DB_PASSWORD      # must match Step 4
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432

# --- Redis (same machine → 127.0.0.1) ---
REDIS_URL=redis://127.0.0.1:6379/0
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
CELERY_BROKER_URL=redis://127.0.0.1:6379/0
CELERY_RESULT_BACKEND=redis://127.0.0.1:6379/1

# --- MinIO ---
MINIO_ENDPOINT_URL=http://127.0.0.1:9000      # how the APP reaches MinIO
MINIO_PUBLIC_ENDPOINT_URL=http://192.168.1.50 # how the BROWSER reaches it (real IP!)
MINIO_BUCKET=mmftp-files
MINIO_ACCESS_KEY=CHOOSE_A_MINIO_KEY          # must match minio.service (Step 5)
MINIO_SECRET_KEY=CHOOSE_A_MINIO_SECRET       # must match minio.service (Step 5)
MINIO_USE_SSL=False
MINIO_REGION=us-east-1

# --- Email (offline → just log emails instead of sending) ---
DJANGO_EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend
DEFAULT_FROM_EMAIL=mmftp@mm.co.in
```

Lock it down (it holds passwords):

```
chmod 600 /opt/mmftp/.env
```

The values you actually decide (everything else stays as-is)

Value	Where it comes from
<code>DJANGO_SECRET_KEY</code>	generate (command above)
<code>SECRETS_ENCRYPTION_KEY</code>	generate (command above)
<code>POSTGRES_PASSWORD</code>	you pick it — must match Step 4
<code>MINIO_ACCESS_KEY</code> / <code>MINIO_SECRET_KEY</code>	you pick them — must match <code>minio.service</code> (Step 5)
The IP <code>192.168.1.50</code>	your real server IP — appears in 3 places: <code>ALLOWED_HOSTS</code> , <code>CSRF_TRUSTED_ORIGINS</code> , <code>MINIO_PUBLIC_ENDPOINT_URL</code>

Three rules that trip people up

1. `127.0.0.1` vs the real IP. Services on the same box reach each other via `127.0.0.1` . But `MINIO_PUBLIC_ENDPOINT_URL` , `ALLOWED_HOSTS` , and `CSRF_TRUSTED_ORIGINS` use the **real server IP** , because that's what a person's browser types. Mixing

these up is the #1 cause of failures.

2. **Passwords must match in two places** — DB password (`.env` ↔ Step 4), MinIO keys (`.env` ↔ `minio.service`).
3. **MINIO_PUBLIC_ENDPOINT_URL must be the exact address users hit** — download links are signed against it; a mismatch makes downloads fail with 403.

STEP 7 — Initialise Django (migrate, static, admin user)

```
cd /opt/mmftp/app
set -a; source /opt/mmftp/.env; set +a # load the env into this shell

/opt/mmftp/venv/bin/python manage.py migrate
/opt/mmftp/venv/bin/python manage.py collectstatic --noinput
/opt/mmftp/venv/bin/python manage.py createsuperuser
```

STEP 8 — nginx reverse proxy

Create `/etc/nginx/sites-available/mmftp` (Ubuntu) — or `/etc/nginx/conf.d/mmftp.conf` (RHEL):

```
server {
    listen 80;
    listen [::]:80;
    server_name _; # or the server IP/hostname

    client_max_body_size 64m; # allow large uploads
    proxy_read_timeout 3600s;
    proxy_send_timeout 3600s;

    # The Django app (Gunicorn).
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_buffering off;
        proxy_request_buffering off;
    }

    # File downloads/previews — presigned links point here, proxy to MinIO.
    # If you change MINIO_BUCKET, change "mmftp-files" below to match.
    location /mmftp-files/ {
        proxy_pass http://127.0.0.1:9000;
        proxy_set_header Host $host;
    }
}
```

Enable + reload:

```
# Ubuntu:
sudo ln -sf /etc/nginx/sites-available/mmftp /etc/nginx/sites-enabled/mmftp
sudo rm -f /etc/nginx/sites-enabled/default
# All:
sudo nginx -t && sudo systemctl reload nginx
```

STEP 9 — The three app services (systemd)

These keep the web app, worker, and scheduler running. WhiteNoise serves static files, so Gunicorn alone is enough behind nginx.

`/etc/systemd/system/mmftp-web.service`:

```

[Unit]
Description=MMFileTransfer web (Gunicorn)
After=network.target postgresql.service redis.service minio.service

[Service]
User=www-data
WorkingDirectory=/opt/mmftp/app
EnvironmentFile=/opt/mmftp/.env
ExecStart=/opt/mmftp/venv/bin/gunicorn mmftp.wsgi:application \
    --bind 127.0.0.1:8000 --workers 3 --timeout 600
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

`/etc/systemd/system/mmftp-worker.service` :

```

[Unit]
Description=MMFileTransfer Celery worker
After=network.target postgresql.service redis.service minio.service

[Service]
User=www-data
WorkingDirectory=/opt/mmftp/app
EnvironmentFile=/opt/mmftp/.env
ExecStart=/opt/mmftp/venv/bin/celery -A mmftp worker -l info
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

`/etc/systemd/system/mmftp-beat.service` :

```

[Unit]
Description=MMFileTransfer Celery beat (scheduler)
After=network.target postgresql.service redis.service

[Service]
User=www-data
WorkingDirectory=/opt/mmftp/app
EnvironmentFile=/opt/mmftp/.env
ExecStart=/opt/mmftp/venv/bin/celery -A mmftp beat -l info
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

The `www-data` user must be able to read `/opt/mmftp`. After Step 7 run: `sudo chown -R www-data:www-data /opt/mmftp` (or use your own service user consistently in all three units).

Start them all:

```

sudo systemctl daemon-reload
sudo systemctl enable --now mmftp-web mmftp-worker mmftp-beat

```

Tuning concurrency — how many users at once

Two settings on the web service control how many requests are handled in parallel. They are the **only** place concurrency is configured (not in the code), so this is the knob to turn if the site ever feels slow:

```
# in mmftp-web.service, the ExecStart line:
--workers N      # N separate app processes – the main lever
--threads T      # threads per worker (helps when requests wait on DB/MinIO)
# total simultaneous requests ≈ N × T
```

Sizing rule of thumb (`workers = 2 × CPU cores + 1`):

Server	Suggested ExecStart flags
2 cores / 4 GB	<code>--workers 5 --timeout 600</code>
4 cores / 8 GB	<code>--workers 9 --threads 2 --timeout 600</code>
8 cores / 16 GB	<code>--workers 17 --threads 2 --timeout 600</code>

Because file uploads/downloads stream straight to MinIO (they don't occupy a web worker for the whole transfer), even a handful of workers serves a large internal team comfortably. To change it:

```
sudo nano /etc/systemd/system/mmftp-web.service # edit --workers / --threads
sudo systemctl daemon-reload
sudo systemctl restart mmftp-web
```

The **Celery worker** has its own separate concurrency for background jobs (email, thumbnails, cleanup) — raise it with `--concurrency N` on the `mmftp-worker.service` ExecStart line if those ever back up.

STEP 10 — Verify

```
# All services running?
systemctl status postgresql redis nginx minio mmftp-web mmftp-worker mmftp-beat

# Health probe (DB + Redis):
curl -s http://127.0.0.1:8000/healthz

# Logs if something is wrong:
journalctl -u mmftp-web -n 80 --no-pager
journalctl -u mmftp-worker -n 80 --no-pager
```

Then open `http://<server-ip>/` in a browser, log in with the admin account from Step 7, and do a **test upload and download** to confirm everything works.

At this point the server can be disconnected from the internet — the app runs fully offline.

Install once, update many — who does what

The big installer runs **once**. After that, every code change is a light, repeatable update — nobody re-runs the installer.

Phase 1 — First install (one time only)

He runs the big installer once:

```
sudo bash deploy/install_no_docker.sh
```

This sets up everything (Postgres, Redis, MinIO, services, `.env`, admin user). It never needs to run again.

Phase 2 — Every update after that (your code → his server)

You (developer), before pushing:

1. Make code changes.
2. ⚠️ If you changed models, run `makemigrations` and **commit the migration files**. The server only applies committed migrations — it never generates them. This is the #1 thing that breaks updates if forgotten.
3. `git push`.

Him (on the server), to deploy your push:

```
cd ~/mmftp          # his working clone
git pull            # get your new code
sudo bash deploy/update_no_docker.sh
```

That update script (lighter than the installer) does only what's needed:

- copies the new code into `/opt/mmftp/app`
- installs any new Python packages
- runs `migrate` (applies your committed migrations)
- runs `collectstatic`
- restarts `mmftp-web`, `mmftp-worker`, `mmftp-beat`

It does **not** touch the `.env`, the database, the stored files, or the credentials — so updates are safe and repeatable.

One setup detail that makes this clean

Tell him to keep his git clone in a normal directory (e.g. `~/mmftp`), **not** in `/opt/mmftp/app`. Reason: `/opt/mmftp/app` is owned by `www-data` and managed by the scripts — running `git pull` directly in it causes ownership/permission headaches. So:

- `~/mmftp` = his working git clone (where he runs `git pull` and the scripts)
- `/opt/mmftp/app` = the live deployed copy (the scripts rsync into it)

Both `install_no_docker.sh` and `update_no_docker.sh` are run from `~/mmftp` and copy into `/opt/mmftp/app` automatically.

Day-to-day operations

```
# Restart after a code change (worker/beat have NO autoreload – always restart):
sudo systemctl restart mmftp-web mmftp-worker mmftp-beat

# Tail logs:
journalctl -u mmftp-web -f

# Backup the database:
sudo -u postgres pg_dump mmftp > mmftp-db-$(date +%F).sql

# Backup the files:
tar -czf mmftp-files-$(date +%F).tar.gz -C /opt/mmftp/minio-data .
```

Applying a code update:

```
cd /opt/mmftp/app
git pull                                # or copy new code in
set -a; source /opt/mmftp/.env; set +a
/opt/mmftp/venv/bin/python manage.py migrate
/opt/mmftp/venv/bin/python manage.py collectstatic --noinput
sudo systemctl restart mmftp-web mmftp-worker mmftp-beat
```

If the server is offline by update time, updating Python packages will need internet again (or a one-off `pip download` bundle). Code-only changes that don't add new packages update fine offline.

Troubleshooting

Symptom	Fix
<code>DisallowedHost</code> error	Add the server IP/hostname to <code>DJANGO_ALLOWED_HOSTS</code> , restart web.
Login fails with CSRF error	<code>CSRF_TRUSTED_ORIGINS</code> must match scheme+host exactly (<code>http://</code> vs <code>https://</code>).
Upload OK but download 403 / <code>SignatureDoesNotMatch</code>	<code>MINIO_PUBLIC_ENDPOINT_URL</code> host $\neq$ the host the browser used. They must match exactly.
Download 404 at <code>/mmftp-files/...</code>	nginx <code>location /mmftp-files/</code> doesn't match <code>MINIO_BUCKET</code> . Align them, reload nginx.
DB auth failure	<code>.env</code> <code>POSTGRES_PASSWORD</code> must match what you set in Step 4.
Postgres "connection refused" on 5432, but <code>postgresql</code> shows active	The cluster is on a non-default port. Run <code>pg_lsclusters</code> — if it shows <code>5433</code> , another service held 5432 at install time. Fix: <code>sudo sed -i 's/^port = 5433/port = 5432/' /etc/postgresql/16/main/postgresql.conf && sudo systemctl restart postgresql@16-main</code> . (The umbrella <code>postgresql.service</code> reports "active" even when the cluster isn't serving — check <code>postgresql@16-main</code> .)
Any service won't start: "Address already in use"	Another program owns that port (6379/9000/5432/80). Find it: <code>sudo ss -ltnp \ grep :PORT</code> . Stop the conflicting service before starting ours. On a clean dedicated server this won't happen.
App can't reach MinIO	MinIO keys in <code>.env</code> must match the <code>minio.service</code> file. <code>systemctl status minio</code> .
Scheduled cleanup/expiry not running	<code>systemctl status mmftp-beat mmftp-worker</code> — both must be running.

Adding HTTPS later

1. Put the internal-CA cert/key on the server (e.g. `/etc/ssl/mmftp/`).
2. Add a `listen 443 ssl;` server block to the nginx config; redirect `:80 → :443`.
3. In `.env`: set `DJANGO_SECURE_SSL=True` and switch `MINIO_PUBLIC_ENDPOINT_URL` + `CSRF_TRUSTED_ORIGINS` to `https://...`.
4. `sudo nginx -t && sudo systemctl reload nginx` and `sudo systemctl restart mmftp-web mmftp-worker mmftp-beat`.